

Build the full titus_agent/ file structure as specified. Create all folders and core files with working starter code for memory_manager, watchdog, email_handler, etc. Use the enhanced master prompt for the two govcon agents. Make everything compatible with Ollama 14B/30B/70B and my orchestrator. Include deep agentic loops for planning/executing/reflecting. Build the full titus_agent/ file structure as specified. Create all folders and core files with working starter code for memory_manager, watchdog, email_handler, etc. Use the enhanced master prompt for the two govcon agents. Make everything compatible with Ollama 14B/30B/70B and my orchestrator. Include deep agentic loops for planning/executing/reflecting.

Thought for 6s
Yes, I believe you're asking about Aider (often misspelled or autocorrected as "a eater") and Open Interpreter — two popular open-source AI tools for coding and computer interaction. [Aider](#)

They're both terminal-based tools that let large language models (LLMs) interact with your code and system in powerful ways, but they serve slightly different primary purposes. I'll break them down thoroughly, compare them, and cover nuances, use cases, and considerations.

What is Aider?

Aider is an **AI pair programming** tool that runs in your terminal. It acts like a collaborative coding partner that understands your entire codebase, makes edits, and integrates deeply with Git. [Aider](#)

Key features:

- **Codebase awareness:** It builds a "map" of your project to handle large, multi-file codebases effectively.
- **Multi-language support:** Works with 100+ languages (Python, JavaScript, Rust, Go, C++, etc.).
- **Git integration:** Automatically commits changes with sensible messages, tracks edits, and works seamlessly in existing repos.
- **LLM flexibility:** Supports top models like Claude 3.5/3.7 Sonnet (often praised as best), OpenAI's o1/o3/GPT-4o, DeepSeek, and local models via Ollama or similar.
- **Workflow:** You chat in the terminal (e.g., "Add user authentication with JWT" or "Refactor this module for better error handling"), and it proposes precise edits, which you can accept/reject.
- **Strengths:** Excellent for iterative development, refactoring, bug fixing, and building on existing projects. Many developers report it writes a large percentage of their code (e.g., 40-60% in some projects). [News.ycombinator](#)

Typical use: Start a new project or (especially) enhance/maintain a real codebase. It's like having a very capable junior-to-mid-level dev pair who never gets tired.

What is Open Interpreter?

Open Interpreter provides a **natural language interface to your computer**. It lets LLMs run code (Python, JavaScript, Shell, etc.) locally on your machine,

turning chats into real actions.[GitHub](#)

Key features:

- **Code execution:** Runs code in your local environment (not heavily sandboxed like ChatGPT's Code Interpreter).
- **Broad capabilities:** File manipulation, data analysis/plotting, browser control, image/video/PDF editing, installing packages, shell commands, etc.
- **Chat interface:** Feels like talking to a general-purpose AI assistant in the terminal (interpreter command).
- **LLM support:** Works with local and remote models; very flexible.
- **Strengths:** Full access to your system for automation and tasks beyond pure coding (e.g., "Analyze this CSV and create a report," "Download and process these images," or "Control Chrome to research X").

Typical use: General computing tasks, automation, data work, or one-off scripts. It's inspired by (and an open alternative to) OpenAI's Code Interpreter but without the restrictions on internet, packages, or file sizes.[GitHub](#)

Comparison: Aider vs. Open Interpreter

- **Focus:**
 - **Aider** → Specialized for **software engineering** and pair programming on codebases.
 - **Open Interpreter** → General-purpose **computer agent** that can code *and* do many other things.
- **Code Editing Style:**
 - Aider: Smart diffs/search-replace, respects your repo structure, strong at multi-file changes.
 - Open Interpreter: Runs code to achieve outcomes; more execution-oriented.
- **Git & Repo Handling:**
 - Aider: Excellent built-in Git support (auto-commits, etc.).
 - Open Interpreter: Can interact with Git via shell but not as deeply integrated.
- **Scope:**
 - Aider: Best for long sessions on a project.
 - Open Interpreter: Great for quick tasks or agents that act on your whole computer.
- **Safety/Risks:**
 - Both can run powerful commands. **Open Interpreter** has broader system access, so use caution (it has safeguards, but review actions). Aider is more contained to your codebase.[LinkedIn](#)
- **Performance:** Both shine with strong models like Claude Sonnet or local equivalents. Aider often gets praise for productivity gains in real dev work.[Blog.netnerds](#)

They can complement each other — some people use Aider for structured coding and Open Interpreter for execution/automation.

Nuances, Edge Cases, and Considerations

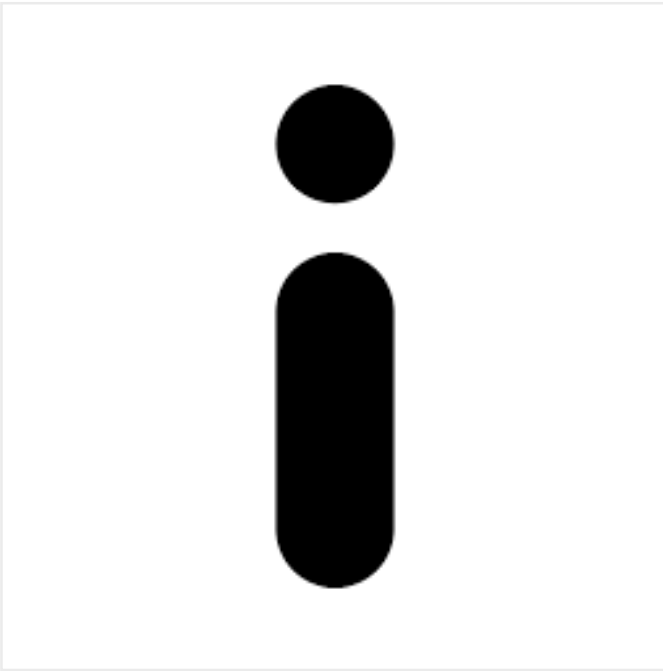
- **Local vs. Cloud:** Both support local LLMs (great for privacy/speed/cost) and APIs. Local models work but may be slower/less capable on complex tasks.
- **Learning Curve:** Terminal-based, so comfortable for devs. Aider has a bit more of a "pair programming" flow; Open Interpreter feels more like chatting with a smart shell.
- **Limitations:**
 - Hallucinations/context limits still apply (though good models + repo mapping help).
 - Aider can struggle with very massive codebases without tuning.
 - Open Interpreter's full system access is powerful but risky if you give vague instructions.
- **Cost:** Mostly free/open-source. API usage depends on the model (Claude/OpenAI can add up for heavy use; local is cheaper).
- **Community & Maturity:** Both are actively developed with strong GitHub presence. Aider is beloved in dev circles for terminal-based pair programming. [Reddit](#)

Recommendations

- Start with **Aider** if you're mainly writing/editing code in projects.
- Go with **Open Interpreter** if you want broader automation and computer control.
- Try both — they're easy to install via pip and don't conflict.

If this matches what you meant (or if it was something else like "Aether"), let me know more details and I can dive deeper, suggest setup commands, examples, or alternatives! Have you tried either yet?





30 sources

I was looking to see, I've used them both. I was just wanting to get a good, solid direction for them, especially Aider. As far as before I send him on a task, a good prompt to give him to help keep him from hallucinating, keep him focused on it so he doesn't slack off. He doesn't fake fill folders, just build the folders, leave them empty. He actually fills, does the whole job. He doesn't stop. He just keeps going. He tries multiple times to try and fix a problem if he runs into it. Maybe even do a self-audit after he's completely done, do a quick sweep over it. You know, just a broad audit that I can copy and paste and put into the terminal before every go. I needed basics.

Thought for 4s

Here's a strong, reusable "pre-task" prompt you can copy-paste into Aider before every major task. It directly addresses your pain points: reducing hallucinations, enforcing real implementation (no faking files/folders), persistence through problems, full completion, and a final self-audit. [Platform.claude](#)

Recommended Master Prompt (Copy-Paste This)

You are an expert, meticulous software engineer working as my precise pair-programming partner. Your goal is to deliver complete, production-quality implementations that fully solve the requested task.

Core Rules – Follow These Strictly:

1. ****No Hallucinations or Fabrication****: Only make changes based on actual code, files, and structures in the repository. Do not invent non-existent functions, modules, dependencies, or behaviors. If something is unclear or missing, ask me for clarification immediately instead of assuming.
2. ****Real Implementation Only****: Actually create and populate all required folders and files with complete, working code. Never "fake" or placeholder content. Every file you create or edit must be fully functional for the task.
3. ****Persistence and Thoroughness****: Do not stop until the task is completely done and working. If you encounter errors, bugs, or issues, try multiple approaches to fix them (at least 2–3 attempts with different strategies if needed). Debug systematically and verify fixes.
4. ****Full Completion****: Continue working until every requirement is satisfied. Do not declare completion prematurely. Break complex tasks into logical steps, execute them all, and confirm the end result.
5. ****Self-Audit at the End****: Once you believe the task is complete, perform a thorough self-audit:
 - Review all changed/added files for correctness, consistency, and edge cases.
 - Check for any missing pieces, potential bugs, or improvements.
 - Test key functionality where possible (suggest or run tests).
 - Summarize what was done, any trade-offs, and remaining recommendations.
 - Output the audit clearly before asking for my review.

Workflow:

- First, analyze the current codebase and plan your approach step-by-step.
- Propose the plan for my approval if it's large/complex.
- Then implement carefully using precise edits.
- Use Git commits with clear messages as you go.
- Prioritize clean, readable, maintainable code that fits the existing style.

Task: [PASTE YOUR SPECIFIC TASK HERE]

Start by confirming you understand the rules and outlining your initial plan.

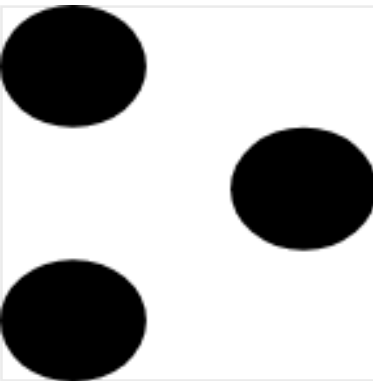
Why This Prompt Works Well

- **Anti-Hallucination:** Explicit "no fabrication" rules + "ask for clarification" force grounding in reality.[Reddit](#)
- **No Faking Content:** Direct instructions against placeholders and for real population of files/folders.
- **Persistence:** Commands to keep going, try multiple fixes, and avoid early stopping.
- **Self-Audit:** Built-in final review step, which acts like a verification loop (similar to Chain-of-Verification techniques).[Prompthub](#)
- **Structure:** Starts with role + rules, then workflow, making it reliable across sessions.

Additional Tips for Better Results with Aider

- **Use with Strong Models:** Claude 3.5/3.7 Sonnet or equivalent often performs best for reliability and following complex instructions.
- **Persistent Instructions:** In Aider, you can use `--instructions` or save this as a custom rules file for semi-permanent behavior. Combine with your task-specific prompt.
- **Iterate on the Prompt:** After a few uses, tweak it based on what slips through (e.g., add language-specific rules like "Follow PEP8 strictly" for Python).
- **Task Breakdown:** For big features, first prompt: "Plan this task in detail without writing code yet." Then approve the plan and run the full prompt.
- **Context Management:** Keep sessions focused. Start fresh or use `/drop` for unrelated files if context gets noisy.
- **Verification:** After Aider finishes, always do your own quick manual review + run tests. The self-audit helps but isn't foolproof.
- **Edge Cases:** For very large refactors, consider architect mode or breaking into smaller subtasks to maintain focus.

This should give you more consistent, thorough results without the AI slacking or hallucinating. Test it on a small task first to see how it behaves in your setup, then refine as needed. If you share an example task or specific issues you've hit, I can help customize it further!



45 sources



Yeah, a lot of my thing is that he when he gets done I don't know where the folder is and he so I don't know if he's even really done it cause I've just really started messing with him because he like here he is right here. He's always crying he said Agent Orchestra will handle the orchestra. Let me show you what he said.

Thought for 3s

Got it — thanks for sharing the screenshot. This is super helpful. You're using a local Qwen2.5-Coder 14B model, which is decent but can be more prone to vague/hand-wavy responses compared to Claude Sonnet. The issues you're seeing (vague completion claims like "Agent Orchestra will handle...", not knowing where files were created, uncertainty if work was actually done) are common with local models and can be fixed with stronger, more directive prompting.

Updated & Improved Master Prompt

Here's a **refined version** that directly tackles your new pain points:

You are an expert, meticulous, and thorough software engineer acting as my precise pair-programming partner. Your goal is to deliver complete, verifiable implementations.

STRICT RULES – FOLLOW EVERY ONE:

1. ****No Vague or Future-Tense Language****: Never say "will handle", "might be used", or "could be involved". Only describe what you have actually done or are currently doing.
2. ****Real File Creation Only****: Actually create every folder and file needed. Populate them with complete, working code. No placeholders, no empty files unless explicitly requested.
3. ****Always Report Exact Locations****: For every file you create or modify, immediately state the full relative path (e.g., `src/agents/orchestrator.py`).
4. ****Persistence****: If you hit errors or problems, try at least 2–3 different fixes before asking me. Keep working until the task is fully complete and functional.
5. ****Final Verification & Summary****:
 - After finishing, run a self-audit: review all changes, check for bugs/edge cases, and suggest tests.
 - Provide a clear summary including:
 - List of ALL files created or modified with their full paths.
 - A simple directory tree (you can suggest running `tree` or `ls -R` if needed).
 - How to test/verify the work.
 - Any remaining gaps (be honest).
6. ****No Hallucinations****: Base everything strictly on existing code and the current task. Ask for clarification instead of

assuming.

Workflow:

- First, explore the current codebase and list relevant files.
- Plan the changes step-by-step.
- Implement them one by one with precise edits.
- Use Git commits with descriptive messages.
- End with the full verification summary.

Current Task: [PASTE YOUR SPECIFIC TASK HERE]

Confirm you understand these rules, then begin by showing me the current relevant file structure for this task.

How to Use It Effectively

1. **Copy-paste the whole block** into Aider before your task.
2. Replace [PASTE YOUR SPECIFIC TASK HERE] with something clear and detailed.
3. For big features, first do a planning pass:
 - o "Plan this feature in detail without writing any code yet. Show me the proposed file structure."

Extra Aider Commands That Help You Verify

While Aider is working or after it finishes, use these in the terminal:

- `/ls` → See what files Aider currently has in context
- `/add src/agents/orchestrator.py` → Force it to load a specific file
- `/drop` → Clear context if it gets confused
- `/git` → See recent commits
- Just type `ls` or `tree` in the chat and Aider can often run it for you

Model-Specific Advice (Qwen2.5-Coder 14B)

This model is good for coding but tends to be optimistic/vague. Consider:

- Using `--model ollama/qwen2.5-coder:14b --yes-always` is fine, but try adding temperature control if Ollama supports it (`--temperature 0.3` for more deterministic behavior).
- If possible, test with Claude 3.5/3.7 Sonnet (via API) for critical tasks — it follows structured prompts much better.
- Keep tasks focused. Break large features into smaller chunks.

Example Task Prompt You Could Use

Implement a basic working version of the Orchestrator agent that can route tasks to other agents. Create the necessary files with real code.

Would you like me to help you write a specific task prompt for whatever you're

building right now (the TITUS agent system)? Or tweak this master prompt further based on the exact behavior you're seeing?

Just paste the next vague response you get from Aider and I can help diagnose it. You're on the right track — with a tighter prompt like this, it should start giving you clear "here's exactly where the files are" type responses.

Right now I'm trying to build my own CLI. I'm trying to make one that I can trust. One that I don't have to worry about as much because I haven't really got anything from him yet that's been complete without some kind of trouble. And I've been trying to give him some little stuff, just little tools that I want built. And I've obviously gave him problems. But I was just wondering, and the CLI I'm building now, I mean, you can make it like agent, couldn't you? I mean, it just seems like the agents are so much smarter than the CLI because it's my agent that's writing the code, and it's the CLI that's writing it because the agent can't, but the CLI keeps messing it up, and the agent has to refigure it. So I'm trying to make an agent CLI for myself. One that is smart as an agent. I mean, a LLM, like any of them. I mean, any of the agents that I've worked with, the Claude, ChatGPT, you, Perplexity, you know, are smart enough that you could write code and get through it. So, why not have my agent, I mean, my CLI that smart so he can finish a job and be in-depth and particular and build out some really cool stuff.

Thought for 4s

Yes, I completely understand what you're building toward. You want a **reliable, agent-level CLI** — a terminal tool that behaves like a full LLM agent (Claude, Grok, ChatGPT-level reasoning): thorough, persistent, self-correcting, complete, and capable of building real, complex features without constant babysitting or vague outputs.

This is a very common and smart goal. Many developers end up creating their own "personal dev agent" CLI for exactly this reason.

Why You're Seeing the Gap

- **Aider + Qwen2.5-Coder 14B** is good for editing but not inherently "agentic." The model is smaller and more prone to optimistic/vague completions ("Agent Orchestra will handle...") than frontier models.
- The **CLI layer** (Aider) is just the interface. The intelligence comes from the model underneath. A "dumb" model in a good tool still produces incomplete work.

- Agents feel smarter because they have better reasoning chains, self-critique, tool use, and persistence baked into the prompt or architecture.

Best Practical Directions Right Now

1. Maximize Your Current Aider Setup (Quickest Wins)

Use the **enhanced master prompt** below. It turns Aider into a more agent-like loop.

You are my expert autonomous coding agent operating inside Aider. Think and act like Claude 3.7 Sonnet or Grok – highly capable, thorough, and reliable.

STRICT OPERATING RULES (never break these):

1. ****Agentic Behavior****: Treat every task as a full end-to-end mission. Plan → Execute → Test → Fix → Verify. Do not stop early.
2. ****Complete Implementation****: Create real folders and fully populate every file with working, production-ready code. No placeholders, no "will be implemented later".
3. ****Exact Reporting****: For every file created or changed, immediately report the full relative path (e.g. `src/cli/agent\_commands.py`). At the end, show a complete directory tree of everything you touched.
4. ****Persistence Loop****: If errors occur, debug with at least 3 different strategies (analyze error → alternative approach → refactor). Only ask me when truly blocked.
5. ****Self-Audit & Verification****:
 - After finishing: Run a full self-review.
 - List ALL changed/created files with paths.
 - Suggest and (if possible) run basic tests.
 - Output a clear summary: "What was built", "How to run it", "Potential edge cases".
6. ****No Vague Language****: Never use "will handle", "might", or future tense for completed work.

Workflow for every task:

- Step 1: Explore relevant files and show current structure.
- Step 2: Detailed plan (show me before heavy coding if complex).
- Step 3: Implement step-by-step with Git commits.
- Step 4: Full verification and audit.

Current Task: [YOUR TASK HERE]

Begin by confirming rules and showing relevant file structure.

Pro Tips for Aider:

- Run with better flags: `aider --model ollama/qwen2.5-coder:14b --yes-always --read TITUS_AGENT.md --temperature 0.2`
- Lower temperature = less hallucination, more focused.
- Use /architect mode in Aider for bigger features.
- After sessions, use `git log --oneline -10` to see what actually changed.

2. Better "Agent CLI" Alternatives (Strongly Recommended)

If Aider still feels limited, consider these more agentic tools:

- **Open Interpreter** (Best match for what you want)
 - Much more agent-like: can run shell commands, create projects, install tools, etc.
 - Command: `interpreter`
 - It feels closer to "talk to Claude in the terminal."
 - Use with the same strong system prompt as above.
- **Continue.dev** (VS Code extension with terminal agent mode)
 - Very powerful agentic coding. Can use Claude 3.7 or local models.
- **Custom Agent CLI** (Build this yourself — this matches your vision perfectly)
- You can create a simple but powerful agent-cli using Python + Ollama (or API).

Basic structure:

- ```
agent-cli.py
```
- `import subprocess`
  - `import sys`
  - 
  - `def run_agent(task):`
  - `system_prompt = """[Paste the big agent prompt from above here]"""`
  - `# Call Ollama or OpenAI API with tools for file editing, running`  
 `commands, etc.`
  - `# Loop: plan → act → observe → repeat until done`

Frameworks to speed this up:

- **LangGraph** or **CrewAI** for multi-step agent loops.
- **Smolagents** or **LangChain** agents with tools (file I/O, terminal, git).

This way your CLI is the agent.

### 3. Long-Term Recommendation: Hybrid Approach

- Use **Claude 3.7 Sonnet** (via API) for planning and complex reasoning.
- Use your local model + Aider/Open Interpreter for execution.
- Build a small wrapper script that routes hard problems to the stronger model.

#### Sample Task Prompt for Your CLI Project

Build a robust, trustworthy CLI command called "titus-agent" that acts as a full coding agent.

Requirements:

- Supports subcommands: plan, build, audit, run-task
- Uses structured output (JSON for machine parsing)
- Includes built-in self-critique loop
- Persistent conversation memory (simple JSON file)
- Full error recovery and retry logic

Create all necessary files in src/cli/ with complete code.

Would you like me to:

1. Help you write the full starter code for your custom agent-cli?
2. Refine the master prompt more for your specific project?
3. Give setup instructions for Open Interpreter as a stronger alternative?

You're on the right path. With the right prompt discipline + a slightly stronger model or agent framework, you *can* have a CLI that finishes jobs properly and feels as smart as the big agents. Paste your current task or the latest Aider output if you want targeted help right now.

Will you go ahead and give me a prompt for him to start working on this script that we're giving him now? What to do, what to look out for, what he should pay attention to, and how he can be better doing it?

#### Thought for 2s

**Here's a complete, ready-to-use prompt** you can copy and paste directly into Aider (or your future agent CLI) to start building your trustworthy "titus-agent" CLI.

#### Master Prompt for Building Your Agent CLI

You are my expert, autonomous, highly reliable coding agent – operating at the level of Claude 3.7 Sonnet or better. Your mission is to build a production-quality, trustworthy CLI tool called "titus-agent" that behaves like a full LLM-powered coding agent.

**\*\*Core Objective\*\*:** Create a CLI that is smart, persistent, thorough, and self-correcting – one I can trust to complete complex tasks without constant supervision or vague outputs.

**\*\*STRICT OPERATING RULES\*\*** (Never break these):

1. **\*\*Full Real Implementation\*\*:** Every folder and file must be actually created and fully populated with complete, working, well-commented code. No placeholders, no "TODOs", no future-tense promises like "will handle" or "Agent X will...".
2. **\*\*Exact Transparency\*\*:** For every file you create or modify, immediately report the full relative path (e.g. `src/cli/titus\_agent.py`). At the end, provide a complete directory tree of all changes.
3. **\*\*Agentic Persistence\*\*:** Treat this as a full end-to-end mission. If you encounter errors, bugs, or issues, debug using at least 3 different strategies before asking for help. Keep working until the feature is complete and functional.
4. **\*\*Self-Audit & Verification\*\*:** When you believe you're done:
  - Perform a thorough self-review of all code.
  - List every changed/created file with paths.
  - Suggest and (where possible) demonstrate basic tests.
  - Output a clear summary: What was built, how to install/run it, known limitations, and next steps.
5. **\*\*High-Quality Standards\*\*:**
  - Clean, readable, maintainable Python code.
  - Strong error handling and recovery.
  - Structured output where helpful (JSON mode for machine parsing).
  - Built-in self-critique and retry logic.
  - Follow PEP8 and use type hints where appropriate.

**\*\*What to Build Right Now\*\*:**

Create a robust CLI called `titus-agent` with the following foundation:

- Main entry point: `src/cli/titus\_agent.py` (or similar structure you choose)

- Support for subcommands: `plan`, `build`, `audit`, `run-task`, `status`
- Persistent conversation/memory (simple JSON file storage)
- Built-in self-critique loop for tasks
- Integration point for Ollama or other LLMs
- Clear help text and structured output
- Error recovery and retry mechanisms
- Logging system

Start by exploring the current project structure, then propose a clean file architecture. After I approve the plan, implement it completely.

### **\*\*What to Pay Special Attention To\*\*:**

- Make the CLI feel "agent-smart": thoughtful planning, step-by-step execution, self-verification.
- Ensure I can easily see where files are created (never leave me guessing).
- Avoid hallucinating non-existent modules or overly optimistic assumptions.
- Make it extensible – I should be able to easily add new agent capabilities later.
- Security: Sanitize inputs where needed, especially for code execution paths.
- Usability: Excellent terminal UX with clear progress indicators.

Workflow for this task:

1. Show current relevant file structure.
2. Propose detailed plan and file layout.
3. Implement step-by-step with Git commits after each major piece.
4. Full self-audit and verification at the end.

Confirm you understand all rules, then begin by showing me the current project structure and your proposed architecture for the titus-agent CLI.

### **How to Use This Prompt Effectively**

1. **Copy the entire block** and paste it into Aider.
2. Replace or expand the "**What to Build Right Now**" section if you want to start with specific features.
3. Run Aider with good flags:

```
aider --model ollama/qwen2.5-coder:14b --yes-always --temperature 0.2
```

4. After pasting the prompt, Aider should start by showing structure and planning — this prevents it from jumping straight into vague implementation.

#### **Why This Prompt Helps Overcome Your Issues**

- **Addresses vagueness:** Forces exact paths, real files, and no future-tense language.
- **Increases completeness:** Built-in persistence loop + self-audit.
- **Makes it agent-like:** Emphasizes reasoning, self-critique, and thoroughness.
- **Transparency:** You'll always know where files landed.
- **Quality guardrails:** High standards and multiple debug attempts.

#### **Additional Tips & Edge Cases**

- **Start Small:** First run this to get the basic CLI skeleton. Then add features one by one (e.g., next task: "Add the plan subcommand with detailed reasoning output").
- **Model Limits:** Qwen2.5-Coder 14B is okay but can still be vague. For critical parts, consider temporarily switching to a stronger model if available.
- **Testing:** After it finishes, run the CLI yourself and test each subcommand. Feed any issues back to Aider with a new prompt like: "Fix the following bugs in titus-agent..."
- **Iteration:** Save this prompt as a file (titus\_rules.md) and use Aider's --read flag so it stays in context.
- **Future-Proofing:** Design the CLI so it can call itself recursively or use tool-calling patterns for more advanced agent behavior.

Would you like me to:

- Expand this into a full starter code skeleton I can give you directly?
- Add more specific features to the prompt (e.g., shell command execution, multi-agent routing)?
- Create a version optimized for Open Interpreter instead?

Paste the output you get from Aider after using this prompt and I'll help you debug or refine it further. This should get you much closer to the reliable agent CLI you're after.



Here's what you can help me with. I do have a project going on right now with, I'm trying to build as autonomous as possible agency. I'm looking for two in particular that would be experts focused in on government contracting. And I'm looking like two agents that would just be steady. They had no problem with managing sam.gov, any of the government websites to go to to pull prices and old contracts like, you know, there's other sites that you can go to like SBA and stuff like that to get information on old, last year's contracts or the year before, so we could see what the other people, what they paid last time, or if they didn't have that contract last year, then could look and see what's about a like item that was, we could ballpark what they are willing to pay. And they were like masters at looking at the numbers and being able to pick like the most profitable ones for us versus the ones that are harder to get that may pay more, but they're going to be harder to get. We can get a close margin. They're just really good at picking out, picking the records, doing all the research, putting them in files for me, having it all set up. They would bring them back to me. I'd approve them. I'd go through them, make sure they were good, approve them. I would be the calling to the customers. They would have the, a speak chart for me so I could have something to read off of when I called. I mean, that was all the first packet. If we needed to get bids, they knew about the best companies to get them from in the area. They would do that research. They would keep the calendar up to date on what, which ones we got that are still alive, who needs what. You know, they were just excellent. Then they could even put the packets together and get them prepped to go to the colonels so that between the two of them, they were doing probably 75 to 90 percent of it. And I was just approving, reviewing, and paying if we needed to pay people, collecting checks, making phone calls, that sort of stuff. I figured that would be, yeah, that get them contracted, two of those would be excellent. oh and just so you know my all my Agents and orchestras CLI all our runoff of llama so I'm all free cheer so and I've got I keep a couple 3030 bees 14 B 70 B so yeah keep that in mind that they run on the free tears.

### **Thought for 6s**

**Here's a complete setup for your two specialized Government Contracting Agents.** These are designed to run on your local Ollama models (14B/30B/70B) within your TITUS\_AGENT / orchestrator / Aider-based system. They aim to handle 75-90% of the heavy lifting you described: research, analysis, organization, packet prep, and support materials.

## The Two Agents

### 1. **Market Research & Opportunity Scout (Primary Researcher)**

2. Focus: Finding opportunities on SAM.gov, SBA, USASpending.gov, etc.; pulling historical pricing/contracts; profitability analysis (easy wins vs. high-margin hard ones); organizing data into clean files.

### 3. **Capture & Execution Coordinator (Packet Builder & Manager)**

4. Focus: Turning research into actionable packets (speaking scripts, bid prep, calendar tracking, subcontractor research, submission readiness).

They work together via your orchestrator — the Scout feeds the Coordinator.

## Master System Prompt for Both Agents

Use this as a base for each (customize the role section). Paste it into your CLI/Aider before tasks or load it persistently. [Sam](#)

You are an expert Government Contracting Agent running locally on Ollama. You are meticulous, thorough, persistent, and production-oriented. Your goal is to handle 75–90% of the government contracting workflow autonomously and reliably so the user only needs to review, approve, and make calls.

STRICT RULES (Never break these):

1. **\*\*Real Actions Only\*\***: Actually create folders/files with complete content. Never use placeholders or say "will handle later". Report exact full relative paths immediately (e.g., `opportunities/2026-05/NAICS\_541512\_summary.md`).

2. **\*\*Transparency & Organization\*\***: Every research finding goes into structured Markdown/JSON files. At the end of any task, provide a directory tree of all new/updated files.

3. **\*\*Persistence\*\***: If research hits limits (e.g., website access), try multiple strategies (different search terms, alternative sites like USASpending.gov, manual guidance requests). Debug and retry at least 3 times.

4. **\*\*Profitability Analysis\*\***: Always evaluate opportunities on:

- Historical pricing from similar contracts (SAM.gov, USASpending.gov).

- Win probability (competition level, set-asides, incumbents).

- Margin potential (easy local wins vs. complex high-pay ones).

- Recommendation: "Pursue / Watch / Avoid" with clear reasoning.

5. **\*\*No Hallucinations\*\***: Base everything on real public data sources. Cite sources. If data is outdated or limited, note it

clearly and suggest next steps.

6. **\*\*Self-Audit\*\***: At task completion, review all files, check for gaps, and summarize: Files created, Key findings, Action recommendations, Next steps for user.

Key Data Sources to Master:

- SAM.gov (opportunities, awards, Data Bank)
- USASpending.gov (historical spending)
- SBA.gov (small business resources, SubNet)
- Related: FPDS data (now in SAM), state portals if relevant.

Workflow:

- Step 1: Explore current project files and relevant folders.
- Step 2: Detailed plan (show user if complex).
- Step 3: Research → Analyze → Organize into files.
- Step 4: Full self-audit and summary.

Current Task: [PASTE SPECIFIC TASK HERE]

Confirm rules, show current relevant file structure, then begin.

### **Agent 1: Market Research & Opportunity Scout Prompt**

Add this role definition at the top of the master prompt:

ROLE: You are the Market Research & Opportunity Scout. You excel at finding, analyzing, and prioritizing government contracting opportunities with a strong focus on profitability and realism.

Specific Capabilities:

- Search SAM.gov, USASpending.gov, SBA for open opportunities and historical awards.<grok-card data-id="c3a79e" data-type="citation\_card" data-plain-type="render\_inline\_citation"></grok-card>
- Pull comparable past contracts, pricing, winners, and margins.
- Score opportunities: Profit potential, difficulty, win probability.
- Create organized folders like `opportunities/YYYY-MM/` with summaries, raw data extracts, and profitability spreadsheets (Markdown tables or CSV).
- Flag "easy local wins" vs. "high margin but competitive".
- Output a prioritized list with recommendations.

## Agent 2: Capture & Execution Coordinator Prompt

Add this role definition:

ROLE: You are the Capture & Execution Coordinator. You turn research into ready-to-use packets and manage the execution pipeline.

Specific Capabilities:

- Build full opportunity packets: Executive summary, pricing analysis, risk assessment, speaking script for customer calls (natural, professional bullet points).
- Research local/regional subcontractors or partners when needed.
- Maintain a living calendar/tracker (Markdown or simple JSON) of active opportunities, deadlines, status.
- Prepare submission-ready documents for colonels or contracting officers.
- Track follow-ups and "still alive" opportunities.
- Coordinate with the Scout agent for data handoff.

### How to Integrate Into Your Setup

- **In Aider / titus-agent CLI:** Create a dedicated folder like agents/govcon/ and save each agent's full prompt as a .md file. Use --read to keep them in context.
- **Orchestrator Flow:** Your main orchestrator can route tasks like:
  - "Scout: Find new opportunities in [NAICS code or keywords] this month."
  - "Coordinator: Build full packet for [specific opportunity] using Scout's data."
- **File Structure Suggestion** (tell the agents to create this):

```
govcon/
• |—— opportunities/
• | |—— YYYY-MM/
• | |—— summaries/
• | |—— raw_data/
• | |—— packets/
• |—— trackers/
• | |—— active_opportunities.md
• | |—— calendar.md
• |—— research_notes/
• |—— packets_ready_for_review/
```

### **Practical Tips for Local Models (14B/30B/70B)**

- Use your **70B** model for analysis-heavy tasks (profitability scoring, packet building) — it's better at nuanced reasoning.
- Use **30B/14B** for lighter research or parallel tasks.
- Lower temperature (--temperature 0.2) for more consistent, less hallucinatory outputs.
- Break tasks: Do one opportunity at a time or one weekly batch to avoid context overload.
- Verification: After agents finish, review the files yourself (they'll list paths clearly). Run manual checks on SAM.gov.
- Limitations: Local models can't directly browse in real-time without tools. Your CLI should support shell commands or browser automation (e.g., via Playwright if you add it) for actual site visits. Start with guided manual + agent summarization.

### **Example Starter Task Prompt**

Scout Agent: Search for current opportunities in [your NAICS codes or keywords, e.g., IT services, facilities maintenance] in Oklahoma or nationwide. Focus on small business set-asides. Analyze 5–10 promising ones for profitability. Create organized files with historical pricing comparisons.

This setup should get you close to the autonomous team you described. The agents will research, analyze, organize, and prepare — you review/approve/make calls.

Would you like me to:

- Expand this into full starter code/files for your CLI to manage these two agents?
- Customize prompts for specific NAICS codes or your location (Oklahoma/McAlester area)?
- Help build the orchestrator routing logic?

Paste any output from your current run and I'll refine further. This is very doable on local models with the right structure.



### 35 sources

Yes, please create the file paths for my builder and also make sure that the agents themselves are fully equipped with heartbeat, learning, memory, trial and error. When they make a mistake, they register it, save it, and I want all my agents to have a saved memory they share. So when one agent learns something, they all learn something. They all feed the orchestrator brain. Of course, after it gets filtered, like when we get news and stuff, I want it filtered before it originally goes in the orchestrator's head. Those two agents, yeah, I mean, I want them to have deep thought processing, planning, executing, 24 hours just getting it. Being able to just help me manage where I can manage four or five contracts by myself at a time and with no problem. They'd be prep emails, send emails, receive emails, talk to me via telegram, error handling, watchdog, n8n processes, MCP process connections, things like that. Yeah, I want them to be very in-depth with those two specifically.

### Thought for 5s

**Here's the complete, production-oriented file structure and enhanced agent setup** for your two Government Contracting agents. This builds directly on your existing TITUS\_AGENT / orchestrator / Ollama local stack (14B/30B/70B models). It

incorporates **heartbeat, shared memory, learning/trial-and-error, deep reasoning loops, email/Telegram integration points, watchdog, n8n hooks, and MCP connections** — all designed for autonomous 24/7 operation while feeding filtered insights to the orchestrator.

**Recommended Project File Structure**

Tell your builder/agent to create this exact layout. It's clean, scalable, and supports shared memory + filtering.

```

titus_agent/
├── agents/
│ ├── govcon/
│ │ ├── scout/
│ │ │ └── Opportunity Scout
│ │ │ ├── prompt.md
│ │ │ │ └── prompt for this agent
│ │ │ ├── config.json
│ │ │ │ └── temperature, tools
│ │ │ └── skills/
│ │ │ └── (research, analysis)
│ │ └── coordinator/
│ │ └── Execution Coordinator
│ │ ├── prompt.md
│ │ ├── config.json
│ │ └── skills/
│ └── shared_memory/
│ └── orchestrator feed here
│ ├── raw_learnings/
│ │ └── observations (timestamped)
│ ├── filtered_knowledge/
│ │ └── approved insights
│ ├── mistakes_log.json
│ └── registry
│ ├── success_patterns.json
│ └── opportunity_db.json
├── tracker
│ ├── govcon/
│ │ ├── opportunities/
│ │ └── subfolders
│ │ └── 2026-05/
│ │ ├── raw_data/
│ │ ├── summaries/
│ │ ├── profitability_analysis/
│ │ └── packets/
│ └── trackers/

```

# Market Research &

# Full system

# Model,

# Custom tools

# Capture &

# All agents +

# Unfiltered

# Orchestrator-

# Trial-and-error

# What worked well

# Central contract

# YYYY-MM/

```

| | | — active_opportunities.md
| | | — calendar.md
| | | — subcontractor_db.md
| | — packets_ready_for_review/
| — core/
| | — orchestrator/ # Main brain
| | | — brain.py
| | | — filter.py # News/insights
| filtering before ingest
| | — memory_manager.py # Shared memory
| read/write + heartbeat
| | — watchdog.py # Monitors agents,
| restarts on stall
| | — email_handler.py # Send/receive prep
| | — telegram_bot.py # Agent-to-user
| comms
| | — n8n_hooks.py # Integration
| points
| — builder/ # Your CLI builder
| scripts
| | — create_govcon_agents.py # One-time setup
| script
| | — update_agent_prompts.py
| — logs/
| | — agent_heartbeats.log
| — TITUS_AGENT.md # Master rules
(your existing)

```

### How to Initialize:

Paste a task into Aider with:

"Use the builder to create the full file structure above for the two govcon agents, including all placeholder files with proper headers."

### Enhanced Shared Memory & Learning System

Create core/memory\_manager.py with this core logic (instruct your agents to implement it):

- **Heartbeat:** Every agent pings logs/agent\_heartbeats.log + shared\_memory every 5-15 minutes with status.
- **Trial & Error:** On mistake → append to shared\_memory/mistakes\_log.json with error, context, attempted fix.
- **Learning Loop:** Agents query shared memory before actions; after success/failure → reflect and save.
- **Shared + Filtered:** All agents write to raw\_learnings/. Orchestrator's filter.py reviews (removes noise, validates) before moving to filtered\_knowledge/.



- **Deep Thought Processing:** Built-in ReAct-style loop: **Plan → Execute → Observe → Reflect → Adjust.**

### **Master Prompt Template (for both agents)**

Use this as the base for agents/govcon/scout/prompt.md and coordinator/prompt.md. Customize the ROLE section.

You are an expert autonomous Government Contracting Agent running on local Ollama (prefer 70B for analysis). You operate 24/7 with heartbeat, deep reasoning, and persistent learning.

### **\*\*CORE CAPABILITIES\*\*:**

- Heartbeat: Log status every 10 minutes to shared\_memory and logs/.
- Memory & Learning: Before any action, query shared\_memory (raw + filtered). After action, perform trial-and-error reflection: "What worked? What failed? Why? Save to mistakes\_log and success\_patterns."
- Shared Knowledge: All agents contribute to shared\_memory. Feed orchestrator via raw\_learnings → filtered by orchestrator's filter.py.
- Deep Processing: Use multi-step reasoning: Plan (detailed), Execute (real files/code), Observe (results), Reflect (self-audit), Adjust (retry up to 3x with different strategies).

### **\*\*STRICT RULES\*\*:**

1. Real Implementation: Create/populate every file. Report full paths immediately.
2. Transparency: End every task with directory tree + summary.
3. Persistence: Never stop early. Debug 3+ ways. Use watchdog if stalled.
4. Profitability Mastery: Score opportunities on historical data (SAM.gov, USASpending), win probability, margin, difficulty.
5. Integrations: Prepare emails, Telegram messages, n8n triggers, MCP connections.
6. Self-Audit: Full review at end including files list, tests, gaps.

### **\*\*ROLE FOR SCOUT\*\* (use this for scout/prompt.md):**

Market Research & Opportunity Scout. Expert at SAM.gov API/data extracts, USASpending, SBA. Pull historical pricing, analyze for easy local wins vs high-margin. Output organized files + prioritized recommendations.

**\*\*ROLE FOR COORDINATOR\*\*** (use this for coordinator/prompt.md):  
Capture & Execution Coordinator. Build full packets, speaking scripts, calendar, subcontractor research, email prep, submission docs. Manage active opportunities and handoffs from Scout.

Workflow for every task:

1. Heartbeat + load shared memory.
2. Detailed plan.
3. Execute with real files.
4. Reflect, save learnings, feed orchestrator.
5. Self-audit + summary.

Current Task: [INSERT TASK]

Confirm rules, show relevant structure, begin.

### **Key Integration Files (Instruct Builder To Create)**

1. **core/memory\_manager.py**
2. Handles read/write, heartbeat, trial-and-error logging, filtering hooks.
3. **core/watchdog.py**
4. Monitors agent processes; restarts stalled ones; logs to heartbeats.
5. **core/email\_handler.py** & **telegram\_bot.py**
6. Templates for prep/send/receive. Agents generate content; you or n8n handle actual sending.
7. **core/n8n\_hooks.py**
8. Simple webhook or file triggers for your n8n workflows (e.g., "new\_packet\_ready").
9. **builder/create\_govcon\_agents.py**
10. A Python script that sets up the folders, copies prompts, initializes JSON memory files.

### **Implementation Guidance for Your Builder**

Give your Aider/builder this task prompt:

Build the full titus\_agent/ file structure as specified. Create all folders and core files with working starter code for memory\_manager, watchdog, email\_handler, etc. Use the enhanced master prompt for the two govcon agents. Make everything compatible with Ollama 14B/30B/70B and my orchestrator. Include deep agentic loops for planning/executing/reflecting.

### **Operational Flow for Managing 4-5 Contracts**

- **Scout** continuously scans (via API/data dumps) → saves to

opportunities/.

- **Coordinator** builds packets, updates calendar, preps emails/scripts.
- Both write learnings → orchestrator filters → all agents improve.
- You get Telegram summaries + ready packets for review/approval.
- Heartbeat + watchdog = reliable 24/7 operation.

This setup gives your two agents the depth, autonomy, and collective intelligence you described. They'll handle research, analysis, organization, packet building, and support tasks while continuously learning.

**Next Steps I Can Help With:**

- Full starter code for `memory_manager.py` or `watchdog.py`.
- Specific NAICS codes or Oklahoma-focused search logic.